

Rapport du Projet OS Gestionnaire de fichier

Fait par:

Leticia ZAID

Amine MOUAICI

Samy Islam OULD-YOUNES

CY Cergy Paris Université

Licence 3 Informatique

Département Informatique

Sous la supervision de : Jean Luc Bourdon

05/ 04/ 2025

Table des matières

1. Choix des structures	3
• Superbloc	3
• Bitmap	3
• Inode	4
• Répertoire	5
• LienSymbolique	6
• SystemeFichier	6
• Fonction allouer_inode	7
• Fonction allouer_bloc	7
4. Gestion de l'Initialisation et de la Persistance du Système de Fichiers	8
• Fonction initialiser_systeme_fichier	8
• Fonction sauvegarder_systeme_fichier	8
• Fonction charger_systeme_fichier	8
5. Création d'un fichier/répertoire	9
• Fonction trouver_inode_par_chemin	9
• Fonction create_file_rep	9
6. Gestion des répertoires et permissions:	9
• Fonction trouver_inode_par_cheminCd	9
• Fonction cd	9
• Fonction afficher_ls_chemin	9
• Fonction verifier_permission	10
• Fonction droit	10
• Fonction pwd	10
7. Manipulation d'un fichier/ répertoire	10
• Fonction open_file	10
• Fonction close_file	10
• Fonction ecrire_fichier	11
• Fonction lire_fichier	11
• Fonction cp	11
• Fonction mv	11
• Fonction supprimer_fichier	12
• Fonction supprimer_repertoire	12
8. Création et manipulation des liens	12
• Fonction creer_lien_symbolique	12
• Fonction resoudre_lien_symbolique	12
• Fonction creer_lien_hard	13

1.Choix des structures

Pour la gestion de notre système de fichiers, nous avons conçu une structure de données en C qui modélise les éléments clés d'un système de fichiers, à savoir le superbloc, les inodes, les répertoires, les fichiers ouverts, et les liens symboliques. Cette approche permet une gestion efficace de l'espace de stockage et des métadonnées tout en assurant des opérations rapides sur les fichiers et les répertoires. Voici un aperçu détaillé des choix de structure :

- Superbloc

```
typedef struct {  
    int taille_fs;  
    int nb_inodes;  
    int nb_blocs;  
    int inode_libres;  
    int blocs_libres;  
    int premier_bloc_libre;  
} Superbloc;
```

-Le superbloc est une structure centrale qui contient les métadonnées globales du système de fichiers :

- *La taille totale du système (taille_fs).
- *Le nombre total de blocs et d'inodes (nb_inodes, nb_blocs).
- *Le nombre de blocs/inodes libres (inode_libres, blocs_libres).
- *Le premier bloc libre disponible (premier_bloc_libre).

-Cette structure est inspirée directement du superbloc dans ext2/ext3, indispensable pour l'initialisation et la cohérence du système. Elle permet de détecter rapidement les incohérences en cas de corruption ou d'erreur et fournit un accès direct à des informations souvent utilisées, sans avoir à parcourir toute la partition.

- Bitmap

```
typedef struct {  
    unsigned char inodes[(MAX_INODES + 7) / 8]; // Aligné sur 8  
    unsigned char blocs[(MAX_BLOCKS + 7) / 8]; // Aligné sur 8
```

```
} Bitmap;
```

```
#define BIT_SET(byte, pos) (byte |= (1 << pos))  
#define BIT_CLEAR(byte, pos) (byte &= ~(1 << pos))  
#define BIT_CHECK(byte, pos) (byte & (1 << pos))
```

-Deux bitmaps sont utilisés :

- * un pour les inodes,
- * un pour les blocs de données.

-Chaque bit indique si l'unité correspondante est libre (0) ou occupée (1).

-Cette approche est très efficace en termes d'espace mémoire et permet des opérations rapides (allocation/libération) à l'aide de macros simples (BIT_SET, BIT_CLEAR, BIT_CHECK).

-La taille de chaque tableau est calculée pour être suffisante pour couvrir le nombre maximal d'inodes et de blocs, en arrondissant au nombre d'octets nécessaires (en utilisant $(MAX_INODES + 7) / 8$ et $(MAX_BLOCKS + 7) / 8$). Les macros BIT_SET, BIT_CLEAR, et BIT_CHECK permettent respectivement de définir, effacer ou vérifier l'état d'un bit donné dans un octet, facilitant ainsi l'allocation et la libération des ressources.

● Inode

```
typedef struct {  
    int id;  
    int taille;  
    int est_repertoire;  
    int permissions;  
    int liens;  
    int est_lien;  
    int blocs[NUM_DIRECT_BLOCKS]; // Blocs directs  
    int indirect_block;           // Bloc indirect  
    int inode_pere;                // Référence à l'inode parent  
    time_t date_creation;  
    time_t date_modification;  
} Inode;
```

-Chaque inode représente un fichier ou un répertoire (est_repertoire) avec :

- *Sa taille, permissions, date de création/modification.
- *Un tableau de blocs directs et un bloc indirect (pour les fichiers plus gros).
- *Un champ lien pour gérer les liens.
- *Une référence à l'inode parent (inode_pere).

-Cette structure est bien adaptée car :

- Elle est modulaire, ce qui permet de représenter aussi bien des fichiers que des répertoires.
- Elle sépare les métadonnées des données, en suivant le modèle Unix.
- Elle permet une gestion directe et indirecte des blocs, offrant ainsi une bonne scalabilité (adaptée aussi bien aux fichiers courts qu'aux fichiers longs).
- Elle permet une gestion fine des droits d'accès, ainsi que de la structure arborescente du système de fichiers.

• Répertoire

```
typedef struct {  
    int inode_id;  
    struct {  
        char nom[NAME_SIZE];  
        int inode_id;  
    } fichiers[MAX_FILES];  
    int nb_fichiers;  
} Repertoire;
```

-Cette structure permet un accès direct et rapide aux fichiers contenus dans un répertoire.

-Assure une navigation rapide et déterministe dans le système de fichiers.

-Cette structure modélise la relation nom <-> inode, comme dans ext2.

- LienSymbolique

```
typedef struct {  
    int inode_id;  
    char cible[MAX_PATH_LENGTH];  
} LienSymbolique;
```

-Cette structure permet de gérer les liens symboliques et associe un inode à un chemin de fichier cible (stocké sous forme de chaîne de caractères).
-C'est une implémentation fidèle au modèle Unix (où le lien symbolique est un fichier spécial contenant un chemin). Cela permet une navigation flexible dans la hiérarchie.

- SystemeFichier

```
typedef struct {  
    Superbloc superbloc;  
    Bitmap bitmap;  
    Inode inodes[MAX_INODES];  
    Repertoire racine;  
    char data[PARTITION_SIZE];  
    int repertoire_courant;  
} SystemeFichier;
```

-Cette structure regroupe toutes les composantes : Superbloc, bitmaps, inodes, racine, tableau de PARTITION_SIZE octets qui simule le disque virtuel...
-Cette structuration centralisée permet une sauvegarde/restauration simple via la sérialisation dans un fichier binaire. Elle facilite également la navigation dans le système et la mise en œuvre des fonctionnalités (mkdir, ls, cd, etc.).

2. Définition des constantes

- **MAX_INODES 100** : Limite le nombre d'inodes à 100, suffisant pour un petit système de fichiers.
- **MAX_BLOCKS 1024** : Fixe à 1024 le nombre de blocs de données, offrant une capacité de stockage modérée.
- **BLOCK_SIZE 4096** : Taille classique de 4 Ko pour un bloc de données, optimale pour les performances.
- **MAX_LINKS 10** : Limite à 10 le nombre de liens physiques par fichier, pour éviter une gestion trop complexe.
- **NUM_DIRECT_BLOCKS 12** : Chaque inode peut pointer vers 12 blocs directs, une configuration courante pour une gestion simple.

- **INDIRECT_BLOCKS 1** : Un bloc indirect par inode pour étendre l'adressage des blocs de données.
- **MAX_FILES 32** : Limite à 32 fichiers par répertoire pour simplifier la gestion des répertoires.
- **NAME_SIZE 32** : Taille de 32 caractères pour le nom des fichiers, suffisant pour la plupart des cas.
- **MAX_PATH_LENGTH 256** : Longueur maximale du chemin fixée à 256 caractères, conforme aux systèmes modernes.
- **MAX_OPEN_FILES 16** : Limite de 16 fichiers ouverts simultanément pour une gestion efficace des ressources.
- **Max_SIZE_PARAM 500** : Limite la taille des paramètres à 500 pour gérer la mémoire de manière raisonnable tout en permettant des entrées suffisamment larges.
- **PARTITION_NAME "mnt.fs"** : Définit le nom du fichier de la partition pour identifier et gérer facilement le stockage.
- **PARTITION_SIZE (MAX_BLOCKS / 4 * BLOCK_SIZE)** : Calcule la taille de la partition comme un quart de la taille totale des blocs, assurant un espace de stockage suffisant et efficace.
- **MODE_READ 0** et **MODE_WRITE 1** : Définissent les modes d'accès (lecture et écriture) en utilisant des valeurs claires et distinctes pour faciliter la gestion des droits d'accès et des opérations sur les fichiers.

3. Fonctions de gestions

- **Fonction allouer_inode**

Cette fonction parcourt le bitmap des inodes pour trouver le premier inode libre (bit à 0). Lorsqu'un inode libre est trouvé, elle le marque comme utilisé en mettant son bit à 1, décrémente le compteur d'inodes libres dans le superbloc, puis retourne l'indice de l'inode alloué. Si aucun inode n'est disponible, elle retourne -1.

- **Fonction allouer_bloc**

Cette fonction cherche un bloc libre dans le bitmap des blocs. Une fois un bloc libre trouvé, elle le marque comme utilisé, met à jour le compteur de blocs libres dans le superbloc, et retourne l'indice du bloc alloué. Si aucun bloc n'est disponible, elle retourne -1.

Ces deux fonctions permettent d'avoir une gestion fine et centralisée de l'espace et les erreurs d'allocation sont gérées proprement pour garantir la fiabilité.

4. Gestion de l'Initialisation et de la Persistance du Système de Fichiers

- Fonction `initialiser_systeme_fichier`

-Cette fonction initialise une nouvelle instance du système de fichiers en :

- Réinitialisant toutes les structures à zéro via `memset`.
- Initialisant les informations du superbloc (taille, nombre d'inodes/blocs, compteurs de blocs/inodes libres).
- Allouant un inode et un bloc pour le répertoire racine.
- Configurant les informations de l'inode racine (permissions, taille, date, etc).
- Initialisant le contenu du répertoire racine avec les entrées spéciales `.` et `..`.
- Marquant l'inode racine comme utilisé dans le bitmap.

-Elle permet de préparer le système de fichiers pour une première utilisation en configurant ses structures essentielles et en créant le répertoire racine.

- Fonction `sauvegarder_systeme_fichier`

-Cette fonction enregistre l'état actuel du système de fichiers dans un fichier binaire nommé `PARTITION_NAME`. Elle utilise `fwrite` pour écrire l'ensemble de la structure `SystemeFichier` dans le fichier. En cas d'erreur d'ouverture ou d'écriture, le programme s'arrête avec un message d'erreur.

-Elle permet la persistance du système de fichiers en sauvegardant son état sur disque.

- Fonction `charger_systeme_fichier`

-Cette fonction tente de charger un système de fichiers existant depuis le fichier `PARTITION_NAME`. Si le fichier n'existe pas, elle affiche un message indiquant qu'un nouveau système sera créé. En cas de succès, elle restaure le système de fichiers dans la mémoire et définit le répertoire courant à la racine. On aura comme retour: 1 si le chargement a réussi, 0 sinon (nouvelle initialisation nécessaire).

-Elle permet de rétablir un système de fichiers précédemment sauvegardé pour permettre une continuité des opérations.

5. Création d'un fichier/répertoire

- Fonction `trouver_inode_par_chemin`

Cette fonction parcourt un chemin (absolu ou relatif) pour retrouver l'inode associé. Elle découpe le chemin en parties, vérifie que chaque élément est un répertoire, gère les cas spéciaux comme `.` et `..`, et retourne l'identifiant de l'inode si le chemin est valide.

- Fonction `create_file_rep`

Cette fonction crée un nouveau fichier ou répertoire à un chemin donné. Elle décompose le chemin pour identifier le répertoire parent, vérifie si le fichier existe déjà, alloue un nouvel inode et un bloc, puis ajoute l'entrée dans le répertoire parent.

6. Gestion des répertoires et permissions:

- Fonction `trouver_inode_par_cheminCd`

Cette fonction permet de retrouver l'inode correspondant à un chemin donné, en analysant les différents composants (`/`, `..`, `.`) du chemin (absolu ou relatif). Elle effectue une navigation dans l'arborescence en vérifiant à chaque étape si le répertoire existe, et retourne l'identifiant de l'inode cible s'il est trouvé, ou `-1` sinon.

- Fonction `cd`

Cette fonction simule la commande `cd` d'un terminal. Elle permet de changer le répertoire courant dans le système de fichiers en fonction d'un chemin fourni. Elle utilise `trouver_inode_par_cheminCd` (ou `resoudre_lien_symbolique`) pour trouver l'inode cible. Elle gère aussi les cas particuliers comme `.` et `..`

- Fonction `afficher_ls_chemin`

Cette fonction permet d'afficher le contenu d'un répertoire donné (similaire à la commande `ls`). Elle parcourt le chemin fourni, vérifie chaque sous-dossier et affiche à la fin la liste des fichiers et sous-dossiers présents, avec leurs permissions, taille, inode.

- **Fonction verifier_permission**

La fonction `verifier_permissions` permet de vérifier si un inode possède les droits nécessaires pour effectuer une opération en lecture ou en écriture. Elle se base sur le champ `permissions` de l'inode et retourne vrai si les permissions sont suffisantes (0755 ou 0777 dans ce cas). Cela permet de contrôler les accès aux fichiers ou répertoires, garantissant ainsi un minimum de sécurité et d'intégrité dans le système de fichiers.

- **Fonction droit**

Cette fonction permet de modifier les droits (permissions) d'un fichier ou d'un répertoire, en utilisant une valeur octale (ex : 755). Elle vérifie la validité de l'entrée, puis modifie les permissions dans l'inode associé au fichier ou dossier concerné.

- **Fonction pwd**

La fonction `pwd` affiche le chemin absolu du répertoire courant. Elle commence par parcourir l'arborescence des répertoires en remontant depuis le répertoire courant jusqu'à la racine. À chaque étape, elle ajoute le nom du répertoire actuel à la chaîne de caractères représentant le chemin complet. La fonction gère aussi le cas particulier de la racine et affiche le chemin complet à la fin. Elle est essentielle pour fournir à l'utilisateur une visualisation du répertoire dans lequel il se trouve.

7. Manipulation d'un fichier/ répertoire

- **Fonction open_file**

La fonction `open_file` permet d'ouvrir un fichier en mode lecture ou écriture, selon le paramètre `mode`. Elle commence par rechercher l'inode correspondant au chemin donné, puis vérifie les permissions d'accès à l'aide de la fonction `verifier_permissions`. Si les conditions sont réunies, elle initialise une entrée dans la table des fichiers ouverts, représentée ici par le tableau `fichiers` de la racine. Cette fonction est essentielle pour la gestion des accès aux fichiers dans le système, et elle permet de contrôler l'ouverture simultanée de plusieurs fichiers.

- **Fonction close_file**

La fonction `close_file` sert à fermer un fichier précédemment ouvert en libérant le descripteur associé dans la table des fichiers ouverts. Elle s'assure d'abord que le descripteur fourni est valide avant de réinitialiser l'entrée

correspondante. Cela permet de libérer une place pour d'autres fichiers et de maintenir la cohérence de la gestion des fichiers ouverts dans le système.

- **Fonction `ecrire_fichier`**

La fonction `ecrire_fichier` permet à un utilisateur d'écrire des données dans un fichier existant. Elle ouvre le fichier en mode écriture, vérifie les permissions, puis lit les données saisies depuis l'entrée standard (jusqu'à la détection de CTRL+X). Elle concatène les nouvelles données avec le contenu déjà présent pour éviter d'écraser l'existant. Si le volume de données dépasse la taille d'un bloc, plusieurs blocs sont alloués. Elle met ensuite à jour la taille et la date de modification du fichier. Cette fonction gère donc à la fois la saisie, l'allocation dynamique de blocs, et la mise à jour des métadonnées.

- **Fonction `lire_fichier`**

La fonction `lire_fichier` permet d'afficher le contenu d'un fichier en lecture seule. Elle commence par résoudre les liens symboliques éventuels, puis vérifie que le chemin ne pointe pas vers un répertoire ou un lien mal résolu. Elle contrôle également les droits de lecture et s'assure que le fichier n'est pas vide avant d'afficher le contenu des blocs de données. Cette fonction met en œuvre une lecture sécurisée et complète, en respectant la structure logique du système de fichiers.

- **Fonction `cp`**

La fonction `cp` permet de copier un fichier ou un répertoire d'un emplacement à un autre. Comme pour `mv`, elle vérifie l'existence des chemins source et destination et assure que la destination est un répertoire valide. Après avoir trouvé la source, elle ajoute une copie du fichier dans le répertoire de destination et met à jour son `inode_parent`. La fonction met ensuite à jour les structures du système de fichiers et sauvegarde les modifications. Cette fonction assure la duplication des fichiers tout en préservant les données originales.

- **Fonction `mv`**

La fonction `mv` permet de déplacer un fichier ou un répertoire d'un emplacement à un autre. Elle vérifie d'abord l'existence des chemins source et destination et s'assure que la destination est un répertoire valide. Ensuite, elle retire l'entrée du fichier du répertoire source et l'ajoute dans le répertoire de destination. La fonction met à jour les métadonnées du fichier (en particulier son `inode_parent`) et sauvegarde les modifications dans le système de fichiers. Ce mécanisme garantit le déplacement des fichiers tout en maintenant l'intégrité des données.

- **Fonction supprimer_fichier**

La fonction `supprimer_fichier` permet de supprimer un fichier à partir de son chemin absolu ou relatif. Elle commence par extraire le nom du fichier et le chemin du répertoire parent, puis localise l'inode correspondant. Si le fichier est trouvé, son inode et le bloc associé sont libérés dans les bitmaps. Ensuite, l'entrée du fichier est supprimée du répertoire parent. Cette fonction garantit la cohérence du système de fichiers en mettant à jour le nombre d'inodes et de blocs libres dans le superbloc.

- **Fonction supprimer_repertoire**

La fonction `supprimer_repertoire` gère la suppression d'un répertoire, que ce soit celui spécifié par un chemin ou le répertoire courant. Si le répertoire contient des fichiers ou sous-répertoires, elle les supprime récursivement avant de supprimer le répertoire lui-même. Une fois vide, le bloc et l'inode du répertoire sont libérés, et l'entrée correspondante est retirée du répertoire parent. Cette approche récursive permet d'assurer une suppression complète et propre d'un répertoire et de son contenu.

8. Création et manipulation des liens

- **Fonction creer_lien_symbolique**

Cette fonction permet de créer un lien symbolique dans notre système de fichiers. Elle commence par extraire le nom du lien et le chemin du répertoire parent à partir du chemin complet fourni. Ensuite, elle vérifie que le nom du lien n'existe pas déjà dans le répertoire parent. Si tout est en ordre, elle alloue un nouvel inode et un bloc de données pour stocker la cible du lien symbolique. L'inode est initialisé avec les attributs nécessaires, et la cible est enregistrée dans le bloc alloué. Enfin, l'entrée du lien est ajoutée dans le répertoire parent. Cette fonction permet ainsi de simuler le comportement d'un lien symbolique tel qu'on le retrouve dans les systèmes Unix.

- **Fonction resoudre_lien_symbolique**

Cette fonction permet de résoudre un lien symbolique, c'est-à-dire de retrouver la véritable cible pointée par ce lien. Elle commence par chercher l'inode associé au chemin fourni. Si celui-ci correspond à un lien symbolique, la fonction lit sa cible dans le bloc de données associé et appelle récursivement la fonction sur ce nouveau chemin. Ce mécanisme permet de suivre les chaînes de liens symboliques jusqu'à atteindre le fichier ou répertoire réel.

Elle est essentielle pour garantir que l'accès à un chemin symbolique mène bien à la ressource finale attendue.

- **Fonction `creer_lien_hard`**

La fonction `creer_lien_hard` permet de créer un lien dur entre un fichier existant et un nouveau chemin cible. Elle vérifie d'abord l'existence du fichier source et s'assure que ce dernier n'est pas un répertoire. Ensuite, elle trouve le répertoire parent de la cible, s'assure qu'il a suffisamment de place, puis ajoute le fichier source dans ce répertoire sous un nouveau nom. La fonction augmente également le compteur de liens de l'inode du fichier source, ce qui permet de suivre les références à ce fichier. Enfin, les changements sont sauvegardés dans le système de fichiers.